FastAPI + Docker

How to Dockerize a FastAPI Application

This topic is extremely important because in real-world Machine Learning and Backend development, you don't just build an API you need a reliable way to **deploy and run it anywhere**.

Docker solves the classic problem:

"It works on my computer, but not on the server."

After this lesson, you'll understand:

- What Docker is
- Why Docker is used with FastAPI
- How to create a Docker image
- How to push it to Docker Hub
- How to pull and run it anywhere

---

1. Recap

So far you've learned:

FastAPI
↓
Create Endpoints
↓
Accept Requests
↓
Validate Data
↓
Serve ML Models

Example:

User
↓
FastAPI API
↓
ML Model
↓
Prediction

Everything works on your laptop.

But what if you want:

- Another developer to run it?
- Deploy it on AWS?
- Deploy it on Azure?
- Deploy it on Render?
- Deploy it on Railway?

This is where Docker comes in.

What is Docker?

Simple Definition

Docker is a tool that packages your application along with everything it needs to run.

Think of it as:

A shipping container for software

Just like a shipping container contains:

- Goods
- Equipment
- Instructions

A Docker container contains:

- Your code
- Python
- Libraries
- Dependencies
- Environment settings

Everything needed to run the application.

Real-Life Analogy

Without Docker:

Developer A Computer

✓ Works

Developer B Computer

❌ Missing package

Server

❌ Different Python version

Lots of problems.

With Docker:

Docker Image

↓

Runs Everywhere

Same environment everywhere.

Why Docker is Important for ML Projects

Suppose your ASR project requires:

Python 3.11

Torch

Transformers

Librosa

FastAPI

Uvicorn
Installing all these manually on every server is painful.
Docker packages everything once.
Then:
Build Once
Run Anywhere

---

2. Setup

Before creating Docker images, install Docker.

Step 1: Install Docker Desktop

Download Docker Desktop from the official website:
Docker Desktop

Step 2: Verify Installation

Open terminal:

```
docker --version
```

Example:
Docker version 28.x.x

Step 3: Verify Docker Engine

```
docker run hello-world
```

If successful:

```
Hello from Docker!
Docker is working correctly.
```

Understanding What We'll Dockerize

Suppose your FastAPI project looks like:

```
project/
│
 ├── app.py
 ├── model.pkl
 ├── requirements.txt
```

---

app.py

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
   return {"message": "Hello Docker"}
```

requirements.txt

```
fastapi
uvicorn
```

Docker will package this entire project.

---

3. Create a Dockerfile

The Dockerfile is the most important file.

What is a Dockerfile?

A Dockerfile is a recipe.
It tells Docker:
What OS to use
Which Python version
Which packages to install
How to start application

Create File

Dockerfile
(No extension)

Example Dockerfile

```
FROM python:3.11

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Understanding Every Line

FROM

FROM python:3.11
Base image.
Meaning:
Start with Python 3.11 environment

WORKDIR

WORKDIR /app
Creates working folder inside container.
Equivalent to:
cd /app

COPY requirements.txt
COPY requirements.txt .
Copies requirements file into container.

RUN
RUN pip install -r requirements.txt
Installs all dependencies.

COPY . .
COPY . .
Copies all project files.

CMD
CMD ["uvicorn","app:app","--host","0.0.0.0","--port","8000"]
Runs FastAPI server.

Why Use 0.0.0.0?
Wrong:
127.0.0.1
Only accessible inside container.
Correct:
0.0.0.0
Accessible from outside container.
This is a very common interview question.

---

4. Building Docker Image
Now Docker reads the Dockerfile and creates an image.

What is a Docker Image?
Think:
Dockerfile = Recipe

Docker Image = Finished Cake

Build Command
docker build -t fastapi-app .

Explanation:
docker build

Build image.

-t fastapi-app
Image name.

.
Current directory.

Check Images
docker images
Example:
REPOSITORY TAG
fastapi-app latest

---

5. **Login to Docker**
Before pushing to Docker Hub:
docker login

Enter:
Username
Password
Successfully authenticated.

What is Docker Hub?
Docker Hub is like GitHub for Docker images.
Official site:
Docker Hub

---

6. **Push Docker Image**
Now upload your image.

Step 1: Tag Image
Example username:
sabtain
Tag image:
docker tag fastapi-app sabtain/fastapi-app:latest

Step 2: Push
docker push sabtain/fastapi-app:latest

Result:

Docker Hub
↓

Stores Image

Now anyone can download it.

---

7. Pull Docker Image

On another machine:
docker pull sabtain/fastapi-app:latest
Docker downloads:

- Code
- Python
- Packages
- Configuration

Everything.

Why This Is Powerful
Without Docker:
Install Python
Install FastAPI
Install Uvicorn
Install Dependencies
Fix Version Problems

With Docker:
docker pull image
Done.

---

8. Run Image Locally

Now start the container.

Command

```
docker run -p 8000:8000 fastapi-app
```

Understanding Port Mapping
HOST:CONTAINER
Meaning:
8000:8000

Host machine port 8000
↓
Container port 8000

Now open:

http://localhost:8000

Response:

```
{
```

```
  "message": "Hello Docker"
}
```

Complete Flow Diagram

FastAPI Project
↓
Create Dockerfile
↓
Build Image
↓
Docker Image
↓
Push to Docker Hub
↓
Pull Anywhere
↓
Run Container
↓
FastAPI API Running

ML Deployment Example

For your Pashto ASR project:
app.py
model.safetensors
requirements.txt
Dockerfile
Docker Image Contains:
FastAPI
Whisper Model
Torch
Transformers
Librosa
All Dependencies
Then:
docker build -t pashto-asr .
Push:
docker push username/pashto-asr
Deploy anywhere.

Common Beginner Mistakes

❌ **Forgetting requirements.txt**
Container builds but app crashes.

❌ **Using localhost inside Docker**
127.0.0.1
Use:
0.0.0.0

❌ **Wrong Port Mapping**
docker run -p 5000:8000
Then access:
http://localhost:5000
not 8000.

❌ **Forgetting to Copy Files**
COPY . .
must exist.

Docker Concepts Summary

Concept	Meaning
Dockerfile	Recipe for building image
Image	Packaged application
Container	Running instance of image
Docker Hub	Cloud storage for images
Build	Create image
Push	Upload image
Pull	Download image
Run	Start container

Think of Docker like food delivery:

Recipe
↓
Cook Food
↓
Package Food
↓
Deliver Anywhere

Similarly:

Dockerfile
↓
Build Image
↓
Package App
↓
Run Anywhere

When Dockerizing a FastAPI application:

1. Create your FastAPI app.
2. Create a requirements.txt.
3. Create a Dockerfile.
4. Build the image:
5. docker build -t fastapi-app .
6. Login:
7. docker login
8. Push to Docker Hub:
9. docker push username/fastapi-app
10. Pull anywhere:
11. docker pull username/fastapi-app
12. Run:
13. docker run -p 8000:8000 fastapi-app

This workflow is the standard foundation for deploying FastAPI APIs, ML inference services, speech-to-text systems, recommendation engines, and production AI applications.